

ANALISIS DAN PERANCANGAN SISTEM

APLIKASI PEMINJAMAN ALAT BERBASIS PHP NATIVE MVC

1. Gambaran Umum Sistem

1.1 Nama Sistem

Sistem Informasi Peminjaman Alat

Sistem ini merupakan aplikasi berbasis web yang digunakan untuk mengelola proses peminjaman dan pengembalian alat secara terkomputerisasi.

Sistem memiliki tiga level pengguna:

1. **Admin**
2. **Petugas**
3. **Peminjam**

Ketiga level tersebut memiliki hak akses yang berbeda sesuai dengan kebutuhan proses bisnis.

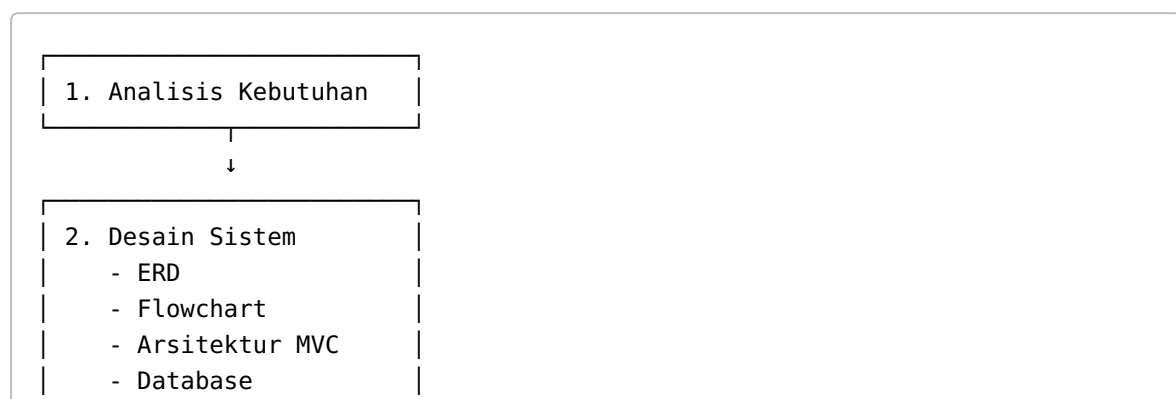
Berdasarkan ketentuan tugas, fitur utama meliputi login/logout, CRUD user, alat, kategori, peminjaman, pengembalian, log aktivitas, persetujuan peminjaman, pemantauan pengembalian, laporan, melihat alat, pengajuan peminjaman, dan pengembalian alat.

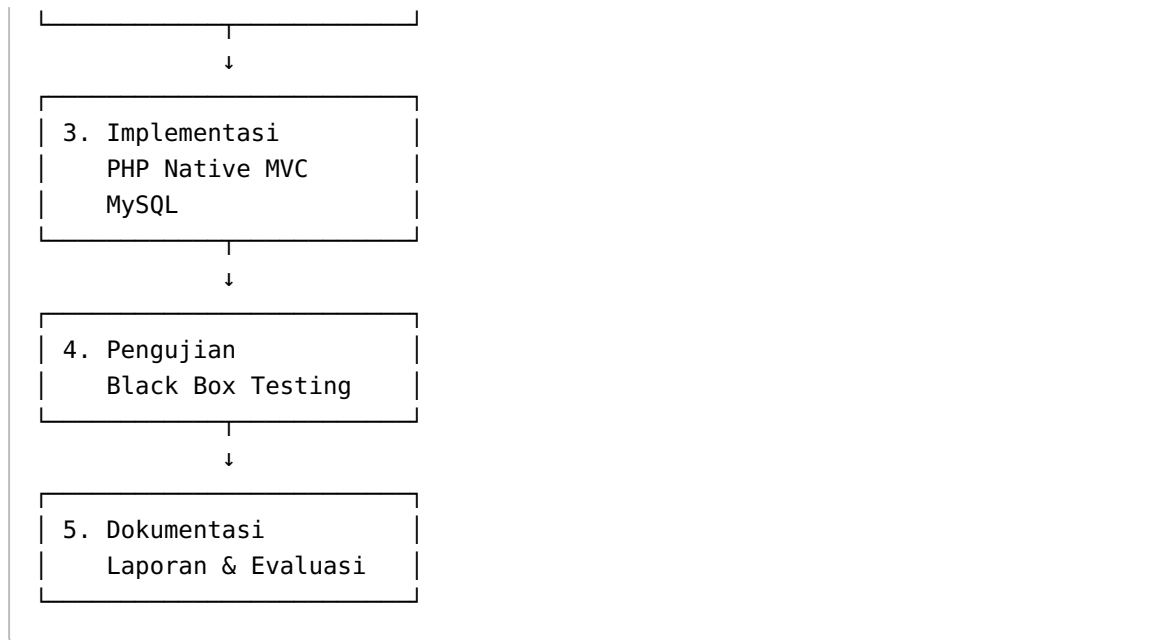
2. Metode Pengembangan Sistem

Metode pengembangan yang digunakan adalah:

WATERFALL

Tahapan Waterfall yang digunakan:





Tahapan ini sesuai dengan langkah kerja pada dokumen tugas yang meminta penggunaan Waterfall sederhana atau prototype dan mencantumkan lima tahap tersebut.

3. ANALISIS KEBUTUHAN

3.1 Identifikasi Aktor

A. Admin

Admin merupakan pengguna dengan hak akses tertinggi.

Tanggung jawab:

- Login
- Logout
- Mengelola user
- Mengelola alat
- Mengelola kategori
- Mengelola data peminjaman
- Mengelola data pengembalian
- Melihat log aktivitas
- Menyetujui peminjaman
- Memantau pengembalian
- Mencetak laporan

B. Petugas

Petugas bertanggung jawab terhadap operasional peminjaman dan pengembalian.

Tanggung jawab:

- Login
- Logout
- Mengelola data peminjaman
- Mengelola pengembalian
- Menyetujui peminjaman
- Memantau pengembalian
- Mencetak laporan

C. Peminjam

Peminjam merupakan pengguna yang menggunakan alat.

Tanggung jawab:

- Login
- Logout
- Melihat daftar alat
- Melihat detail alat
- Mengajukan peminjaman
- Melihat status pengajuan
- Mengembalikan alat

4. Matriks Hak Akses

Modul	Admin	Petugas	Peminjam
Login	✓	✓	✓
Logout	✓	✓	✓
Dashboard	✓	✓	✓
CRUD User	✓	-	-
CRUD Alat	✓	-	-
CRUD Kategori	✓	-	-
CRUD Peminjaman	✓	✓	-
CRUD Pengembalian	✓	✓	-
Log Aktivitas	✓	-	-
Persetujuan Peminjaman	✓	✓	-
Monitoring Pengembalian	✓	✓	-
Cetak Laporan	✓	✓	-
Daftar Alat	✓	✓	✓

Modul	Admin	Petugas	Peminjam
Ajukan Peminjaman	-	-	✓
Pengembalian Alat	-	✓	✓

Matriks ini mempertahankan pembagian fitur Admin, Petugas, dan Peminjam yang tercantum pada soal.

5. Kebutuhan Fungsional

FR-01 Login

Sistem harus menyediakan autentikasi berdasarkan:

- Username/email
- Password
- Role

Sistem melakukan validasi terhadap kredensial pengguna.

Jika benar:

Login → Validasi → Session → Dashboard

Jika salah:

Login → Validasi → Gagal → Pesan Error

FR-02 Logout

Pengguna dapat keluar dari sistem.

Proses:

```

Klik Logout
  ↓
Destroy Session
  ↓
Hapus Session User
  ↓
Redirect Login
  
```

FR-03 Manajemen User

Admin dapat:

- Menambahkan user
- Melihat user
- Mengubah user
- Menghapus user
- Mengubah status user
- Menentukan role user

Data user minimal:

- ID
 - Nama
 - Username
 - Email
 - Password
 - Role
 - Status
 - Created At
 - Updated At
-

6. Manajemen Kategori

Admin dapat melakukan:

Create

Menambahkan kategori.

Read

Melihat daftar kategori.

Update

Mengubah kategori.

Delete

Menghapus kategori jika belum digunakan oleh alat.

Contoh kategori:

Elektronik
Peralatan Praktik
Peralatan Bengkel

Peralatan Laboratorium
Peralatan Komputer

7. Manajemen Alat

Data alat terdiri dari:

- ID alat
- Kode alat
- Nama alat
- Kategori
- Deskripsi
- Jumlah
- Stok tersedia
- Kondisi
- Lokasi
- Foto
- Status
- Created At
- Updated At

Status alat:

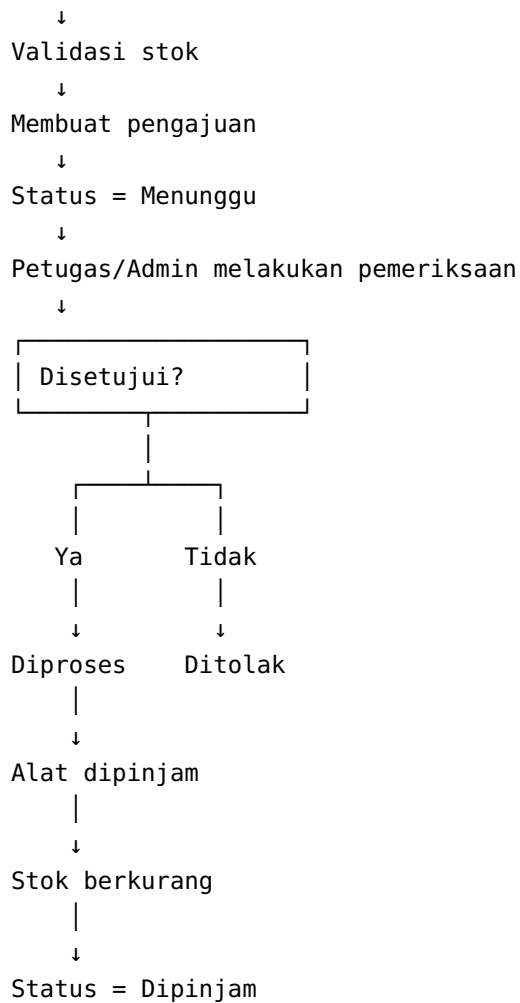
Tersedia
Dipinjam
Rusak
Maintenance
Tidak Aktif

Sistem harus memastikan jumlah alat yang tersedia tidak boleh menjadi negatif.

8. Proses Peminjaman

Proses bisnis utama:

```
Peminjam
↓
Login
↓
Melihat daftar alat
↓
Memilih alat
↓
Mengisi form peminjaman
```



9. Status Peminjaman

Untuk aplikasi yang lebih kompleks, gunakan state berikut:

```
DRAFT
↓
MENUNGGU
↓
DISETUJUI
↓
DIPINJAM
↓
DIKEMBALIKAN
↓
SELESAI
```

Jika ditolak:

MENUNGGU



DITOLAK

Jika dibatalkan:

MENUNGGU



DIBATALKAN

10. Pengembalian Alat

Peminjam melakukan pengembalian alat.

Petugas memeriksa:

1. Tanggal pengembalian
2. Kondisi alat
3. Jumlah alat
4. Keterlambatan
5. Denda jika diperlukan

Alur:

Peminjam mengajukan pengembalian



Petugas menerima alat



Pemeriksaan alat



Hitung keterlambatan



Hitung denda



Update pengembalian



Update stok



Status peminjaman = SELESAI

11. Perhitungan Denda

Denda hanya dikenakan apabila alat terlambat dikembalikan.

Misalnya:

```
Tanggal Jatuh Tempo = 10 September
Tanggal Kembali      = 13 September
```

```
Keterlambatan = 3 hari
Tarif Denda    = Rp5.000 / hari
```

```
Denda = 3 × Rp5.000
       = Rp15.000
```

Rumus:

```
keterlambatan = tanggal_pengembalian - tanggal_jatuh_tempo
```

```
jika keterlambatan > 0:
    denda = keterlambatan × tarif_denda
jika tidak:
    denda = 0
```

Sebaiknya tarif denda disimpan dalam database, bukan hard-code.

12. Log Aktivitas

Karena soal secara eksplisit meminta fitur Log Aktivitas, sistem dibuat menggunakan tabel:

```
activity_logs
```

Data yang dicatat:

- User
- Aktivitas
- Modul
- ID data
- IP Address
- User Agent
- Waktu
- Detail aktivitas

Contoh:

```
Admin
INSERT
alat
```

ID: ALT-001
2026-09-13 19:30:12

Log dapat digunakan untuk audit sistem.

13. Laporan

Sistem menyediakan laporan:

Laporan Alat

Menampilkan:

- Kode alat
- Nama alat
- Kategori
- Jumlah
- Stok
- Kondisi

Laporan Peminjaman

Filter:

- Tanggal
- Peminjam
- Status
- Alat

Laporan Pengembalian

Menampilkan:

- Peminjam
- Alat
- Tanggal pinjam
- Jatuh tempo
- Tanggal kembali
- Keterlambatan
- Denda
- Kondisi kembali

Laporan Aktivitas

Menampilkan:

- User
- Aktivitas
- Modul

- Waktu

Soal juga mensyaratkan adanya pencetakan laporan.

14. Kebutuhan Non-Fungsional

14.1 Performance

Aplikasi harus:

- Menggunakan query yang efisien
- Menghindari looping yang tidak diperlukan
- Menggunakan pagination
- Menggunakan LIMIT untuk data besar
- Menggunakan index database

Ketentuan ini secara langsung tercantum dalam soal.

14.2 Security

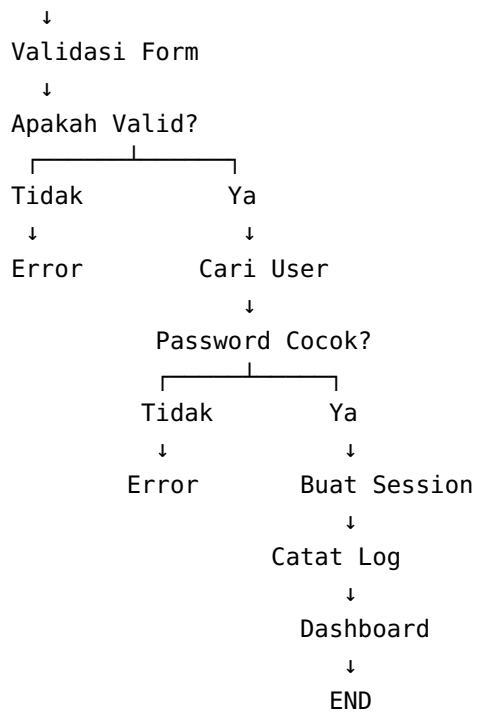
Karena menggunakan PHP Native MVC, keamanan harus diperhatikan:

- Password menggunakan `password_hash()`
 - Login menggunakan session
 - Prepared statement PDO
 - CSRF Token
 - Validasi input
 - Sanitasi output menggunakan `htmlspecialchars()`
 - Role-based access control
 - Session regeneration setelah login
 - Validasi upload file
 - Pembatasan ekstensi file
 - Pembatasan ukuran upload
 - Pencegahan SQL Injection
 - Pencegahan XSS
-

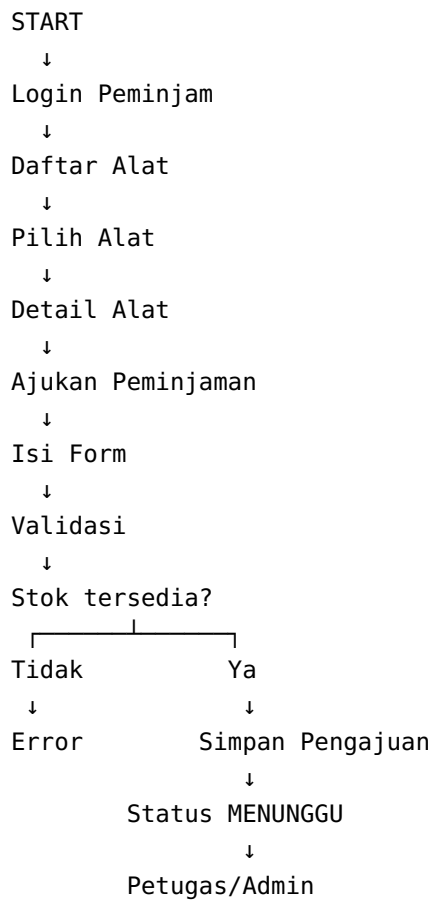
15. BUSINESS FLOW UTAMA

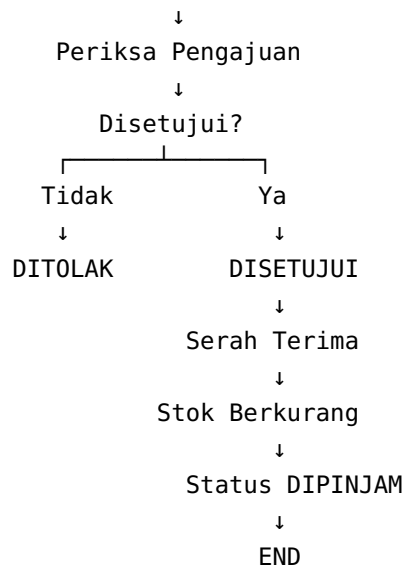
15.1 Flow Login

```
START
↓
Halaman Login
↓
Input Username & Password
```

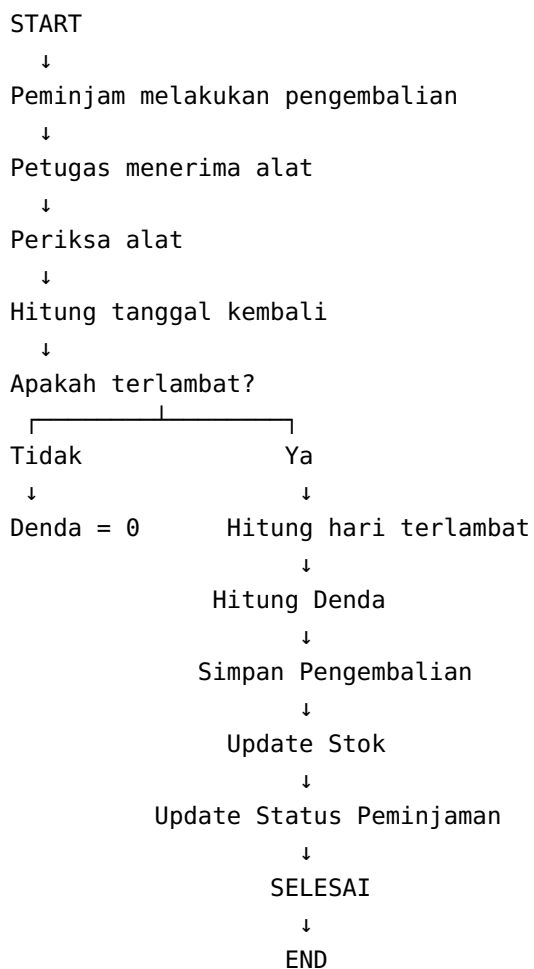


16. FLOWCHART PEMINJAMAN





17. FLOWCHART PENGEMBALIAN + DENDA



Soal secara khusus meminta deskripsi/diagram minimal untuk proses login, peminjaman, serta pengembalian dan perhitungan denda.

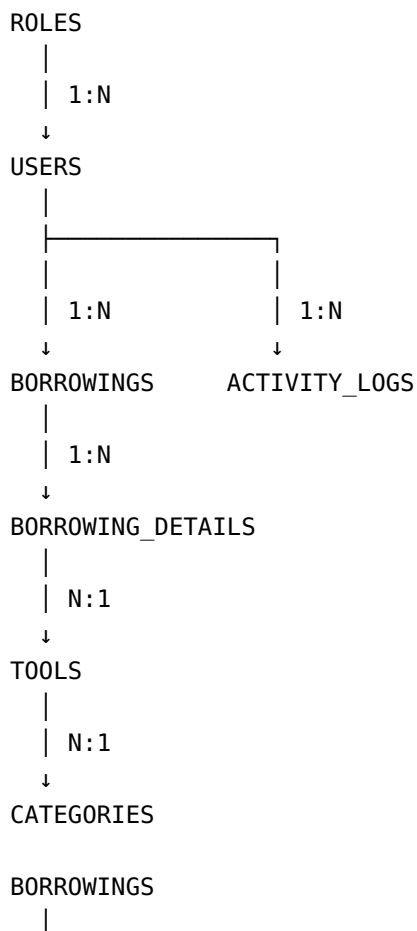
18. ERD

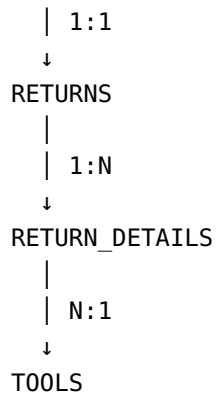
18.1 Entitas Utama

Database dibuat dengan entitas:

```
users
roles
categories
tools
borrowings
borrowing_details
returns
return_details
activity_logs
```

Untuk sistem yang lebih kompleks, saya menyarankan struktur berikut:





19. Struktur Database

19.1 roles

```

roles
-----
id          PK
name
description
created_at
updated_at
  
```

Contoh:

```

1 | Admin
2 | Petugas
3 | Peminjam
  
```

19.2 users

```

users
-----
id          PK
role_id     FK
name
username    UNIQUE
email       UNIQUE
password
phone
address
status
  
```

```
last_login
created_at
updated_at
```

Relasi:

```
roles 1 — N users
```

20. categories

```
categories
-----
id          PK
name
description
status
created_at
updated_at
```

21. tools

```
tools
-----
id          PK
category_id FK
code        UNIQUE
name
description
quantity
available_qty
condition
location
image
status
created_at
updated_at
```

Relasi:

```
categories 1 — N tools
```

22. borrowings

```
borrowings
-----
id          PK
code        UNIQUE
user_id     FK
approved_by FK
borrow_date
due_date
purpose
status
notes
approved_at
created_at
updated_at
```

23. borrowing_details

```
borrowing_details
-----
id          PK
borrowing_id FK
tool_id     FK
quantity
notes
```

Relasi:

```
borrowings 1 — N borrowing_details
tools      1 — N borrowing_details
```

Dengan desain ini satu transaksi peminjaman dapat berisi banyak alat.

Contoh:

```
Peminjaman PJ-001

Laptop      2
Proyektor   1
Kabel HDMI  2
```

Ini lebih baik daripada memasukkan satu alat langsung ke tabel peminjaman.

24. returns

```
returns
-----
id                PK
borrowing_id      FK
received_by       FK
return_date
late_days
total_fine
condition_status
notes
created_at
updated_at
```

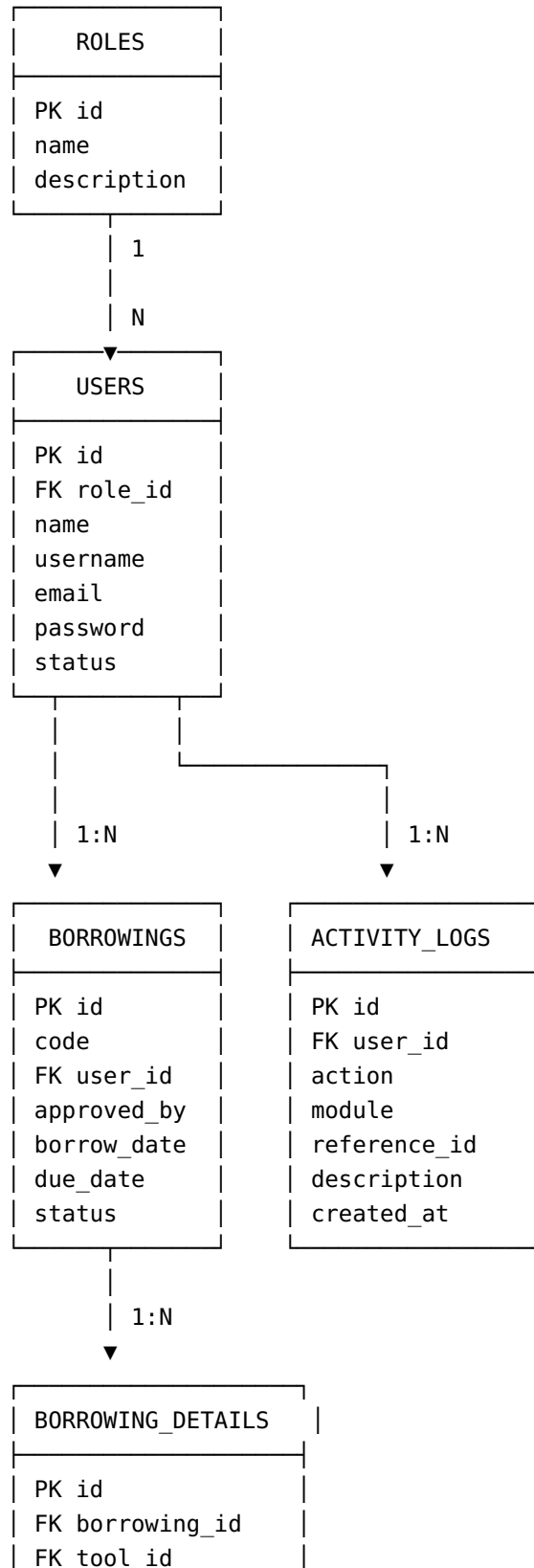
25. return_details

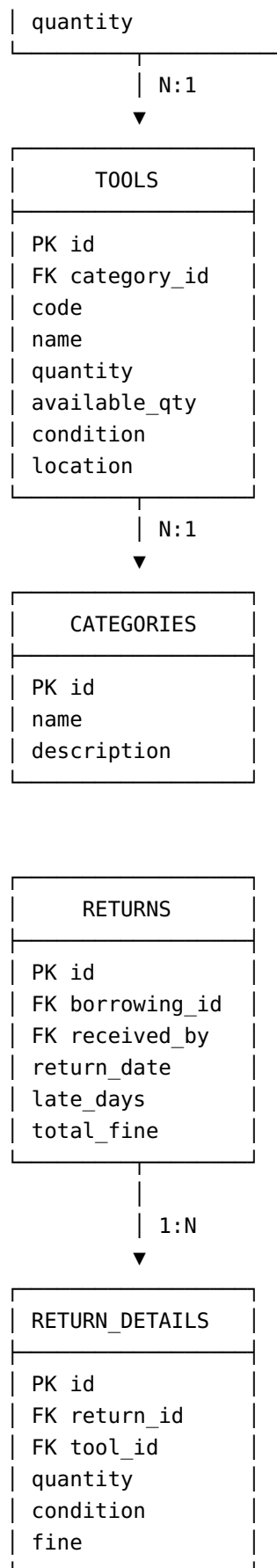
```
return_details
-----
id                PK
return_id         FK
tool_id           FK
quantity
condition
damage_note
fine
```

26. activity_logs

```
activity_logs
-----
id                PK
user_id           FK
action
module
reference_id
description
ip_address
user_agent
created_at
```

27. ERD RELASIONAL





28. Arsitektur PHP Native MVC

Karena aplikasi akan dibuat menggunakan PHP Native MVC, struktur project disarankan:

```
peminjaman-alat/
├── app/
│   ├── config/
│   │   ├── config.php
│   │   ├── database.php
│   │   └── constants.php
│   ├── controllers/
│   │   ├── AuthController.php
│   │   ├── DashboardController.php
│   │   ├── UserController.php
│   │   ├── RoleController.php
│   │   ├── CategoryController.php
│   │   ├── ToolController.php
│   │   ├── BorrowingController.php
│   │   ├── ReturnController.php
│   │   ├── ReportController.php
│   │   └── ActivityLogController.php
│   ├── models/
│   │   ├── User.php
│   │   ├── Role.php
│   │   ├── Category.php
│   │   ├── Tool.php
│   │   ├── Borrowing.php
│   │   ├── BorrowingDetail.php
│   │   ├── ReturnModel.php
│   │   ├── ReturnDetail.php
│   │   └── ActivityLog.php
│   ├── views/
│   │   ├── layouts/
│   │   │   ├── header.php
│   │   │   ├── sidebar.php
│   │   │   ├── navbar.php
│   │   │   └── footer.php
│   │   ├── auth/
│   │   ├── dashboard/
│   │   ├── users/
│   │   ├── categories/
│   │   └── tools/
```

```
|
|
|   ├── borrowings/
|   ├── returns/
|   ├── reports/
|   └── activity_logs/
|
|   ├── core/
|   |   ├── Controller.php
|   |   ├── Model.php
|   |   ├── Database.php
|   |   ├── Router.php
|   |   ├── Session.php
|   |   ├── Auth.php
|   |   ├── Middleware.php
|   |   ├── Validator.php
|   |   └── Response.php
|   |
|   ├── middleware/
|   |   ├── AuthMiddleware.php
|   |   ├── AdminMiddleware.php
|   |   └── PetugasMiddleware.php
|   |
|   └── helpers/
|       ├── url_helper.php
|       ├── date_helper.php
|       ├── security_helper.php
|       └── report_helper.php
|
|   ├── public/
|   |   ├── index.php
|   |   ├── assets/
|   |   |   ├── css/
|   |   |   ├── js/
|   |   |   └── images/
|   |   └── uploads/
|   |
|   ├── routes/
|   |   └── web.php
|   |
|   ├── storage/
|   |   ├── logs/
|   |   └── cache/
|   |
|   ├── database/
|   |   ├── migrations/
|   |   ├── seeders/
|   |   ├── procedures/
|   |   ├── functions/
|   |   ├── triggers/
|   |   └── database.sql
|   |
|   └── README.md
```

29. Konsep MVC

Model

Bertanggung jawab terhadap:

- Database
- Query
- CRUD
- Relasi data
- Transaksi database

Contoh:

```
Tool.php  
Borrowing.php  
ReturnModel.php
```

View

Bertanggung jawab terhadap tampilan:

```
dashboard  
form  
table  
detail  
laporan
```

Controller

Menjadi penghubung Model dan View.

Contoh:

```
BorrowingController  
  
index()  
create()  
store()  
show()  
approve()  
reject()  
cancel()
```

30. Alur Request PHP MVC

```
Browser
↓
public/index.php
↓
Router
↓
Middleware
↓
Controller
↓
Model
↓
Database
↓
Model
↓
Controller
↓
View
↓
Browser
```

Contoh:

```
GET /peminjaman
↓
Router
↓
BorrowingController@index()
↓
Borrowing Model
↓
Database
↓
Data Peminjaman
↓
View borrowings/index.php
```

31. TRANSACTION DATABASE

Proses peminjaman dan pengembalian sangat disarankan menggunakan transaction.

Peminjaman

```
START TRANSACTION;  
  
-- cek stok  
-- insert borrowing  
-- insert borrowing_detail  
-- update stok  
-- insert activity log  
  
COMMIT;
```

Jika terjadi kesalahan:

```
ROLLBACK;
```

Contoh konsep:

```
BEGIN TRANSACTION  
  ↓  
Cek stok  
  ↓  
Stok cukup?  
  ↓  
Insert transaksi  
  ↓  
Insert detail  
  ↓  
Update stok  
  ↓  
Insert log  
  ↓  
Berhasil?  
├── Ya  
│   ↓  
│   COMMIT  
└── Tidak  
    ↓  
    ROLLBACK
```

Soal memang meminta penggunaan operasi relasional, stored procedure, function, trigger, serta `COMMIT` dan `ROLLBACK`.

32. DATABASE TRIGGER

Trigger dapat digunakan untuk audit dan sinkronisasi data.

Contoh penggunaan:

Trigger setelah insert borrowing_detail

Digunakan untuk mengurangi stok alat ketika transaksi peminjaman benar-benar dikonfirmasi.

Namun untuk menjaga konsistensi bisnis, trigger sebaiknya tidak menggantikan seluruh business logic PHP.

Arsitektur yang lebih aman:

```
Controller
  ↓
Service/Business Logic
  ↓
Transaction
  ↓
Database
  ↓
Trigger untuk audit/integritas tertentu
```

33. STORED FUNCTION

Contoh function database:

```
fn_hitung_denda(
    tanggal_jatuh_tempo,
    tanggal_kembali,
    tarif_denda
)
```

Output:

jumlah denda

Konsep:

```
tanggal kembali
  ↓
bandingkan tanggal jatuh tempo
  ↓
terlambat?
├── Tidak
└── Ya
```

↓ ↓
0 hari × tarif

34. CONTROL PROGRAM

Control program utama:

IF

Digunakan untuk:

```
Validasi login  
Validasi stok  
Validasi role  
Validasi denda  
Validasi kondisi alat
```

LOOP

Digunakan untuk:

```
Daftar alat  
Detail peminjaman  
Detail pengembalian  
Laporan
```

SWITCH

Digunakan untuk:

```
Role user  
Status peminjaman  
Status alat
```

Contoh:

```
switch(role)  
  
case ADMIN:  
    dashboard admin  
    break  
  
case PETUGAS:  
    dashboard petugas
```

```
break
```

```
case PEMINJAM:  
    dashboard peminjam  
break
```

35. MODUL SISTEM

Aplikasi dibagi menjadi beberapa modul:

1. Authentication
2. Dashboard
3. User Management
4. Category Management
5. Tool Management
6. Borrowing Management
7. Approval Management
8. Return Management
9. Fine Management
10. Activity Log
11. Reporting
12. Profile
13. System Settings

Untuk versi kompleks, modul dapat dikembangkan menjadi:

Authentication

- |
- |— Login
- |— Logout
- |— Session
- |— Authorization

Master Data

- |
- |— Users
- |— Roles
- |— Categories
- |— Tools

Transaction

- |
- |— Borrowing
- |— Approval
- |— Return

Monitoring

- |
- |— Active Borrowing
- |— Overdue
- |— Activity Log

Report

- |
- |— Tool Report
- |— Borrowing Report
- |— Return Report
- |— Fine Report

36. DASHBOARD ADMIN

Dashboard Admin dapat dibuat kompleks dengan:

DASHBOARD		
Total Alat	Peminjaman	Menunggu Approval
120	35	8

Grafik Peminjaman Bulanan

Peminjaman Terbaru			
PJ-001	Laptop	Menunggu	
PJ-002	Proyektor	Disetujui	

37. DASHBOARD PETUGAS

Menampilkan:

- Peminjaman menunggu approval
- Peminjaman aktif
- Alat terlambat
- Pengembalian hari ini

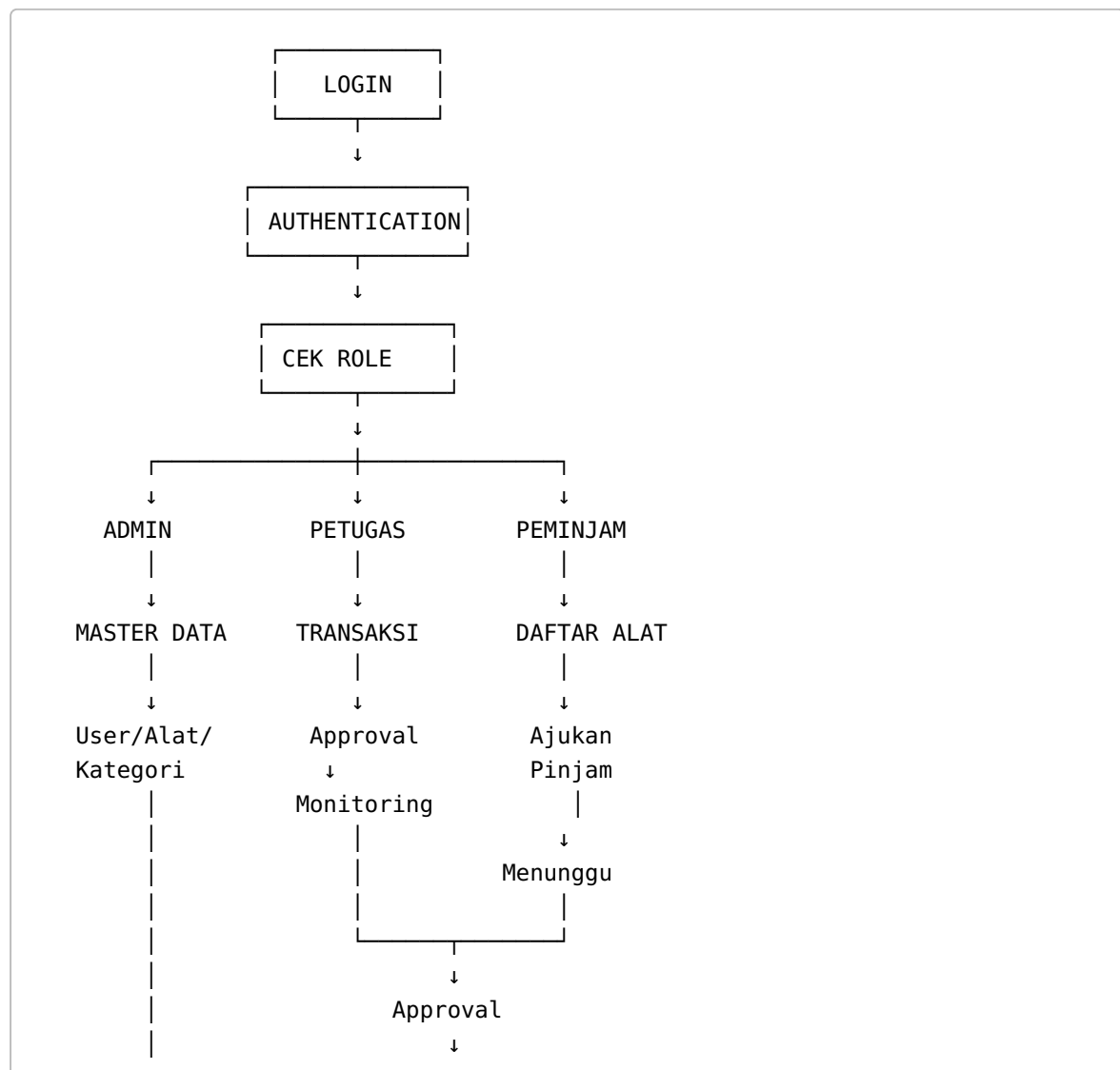
- Total transaksi
- Notifikasi

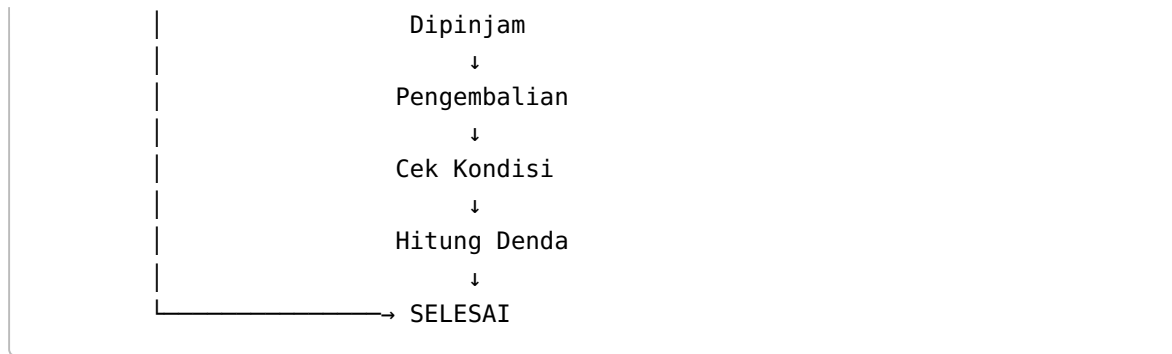
38. DASHBOARD PEMINJAM

Menampilkan:

- Jumlah peminjaman
- Peminjaman aktif
- Peminjaman menunggu
- Peminjaman selesai
- Total denda
- Daftar alat yang tersedia

39. FLOW BISNIS LENGKAP





40. DOKUMENTASI INPUT - PROSES - OUTPUT

Soal meminta dokumentasi setiap modul/proses yang menjelaskan input, proses, output, fungsi, prosedur dan method.

Contoh format:

Modul Login

Input

username
password

Process

Validasi input
↓
Cari user
↓
Verifikasi password
↓
Set session
↓
Catat activity log

Output

Berhasil → Dashboard
Gagal → Pesan error

Method

```
AuthController::login()
AuthController::logout()
User::findByUsername()
Auth::attempt()
```

41. MODUL PEMINJAMAN

Input

```
user_id
tool_id
quantity
borrow_date
due_date
purpose
notes
```

Process

```
Validasi
↓
Cek stok
↓
Buat transaksi
↓
Buat detail
↓
Update stok
↓
Catat log
```

Output

```
Nomor transaksi
Status peminjaman
```

Method

```
BorrowingController::create()
BorrowingController::store()
BorrowingController::show()
```

```
BorrowingController::approve()
BorrowingController::reject()

Borrowing::create()
Borrowing::addDetail()
Borrowing::updateStatus()
```

42. MODUL PENGEMBALIAN

Input

```
borrowing_id
tool_id
quantity
return_date
condition
notes
```

Process

```
Cari peminjaman
↓
Validasi
↓
Hitung keterlambatan
↓
Hitung denda
↓
Simpan pengembalian
↓
Update stok
↓
Update status transaksi
↓
Catat log
```

Output

```
Data pengembalian
Hari keterlambatan
Total denda
Status transaksi
```

Method

```
ReturnController::create()
ReturnController::store()
ReturnController::calculateFine()

ReturnModel::create()
ReturnModel::calculateLateDays()
ReturnModel::calculateFine()
```

43. PENGUJIAN

Metode pengujian menggunakan **Black Box Testing**.

Soal mensyaratkan minimal lima skenario pengujian: login user, tambah alat, pinjam alat, kembalikan alat, dan pengecekan privilege user.

Test Case 1 — Login

ID	Skenario	Input	Expected Result
TC-001	Login benar	Username + password benar	Berhasil masuk
TC-002	Password salah	Password salah	Login ditolak
TC-003	User nonaktif	Akun nonaktif	Login ditolak

Test Case 2 — Tambah Alat

ID	Skenario	Expected
TC-004	Data valid	Alat berhasil ditambahkan
TC-005	Kode duplikat	Sistem menolak
TC-006	Data kosong	Validasi muncul

Test Case 3 — Peminjaman

ID	Skenario	Expected
TC-007	Stok cukup	Pengajuan berhasil
TC-008	Stok tidak cukup	Pengajuan ditolak
TC-009	Quantity 0	Validasi gagal

Test Case 4 — Pengembalian

ID	Skenario	Expected
TC-010	Tepat waktu	Denda Rp0
TC-011	Terlambat	Denda dihitung
TC-012	Alat rusak	Kondisi tercatat

Test Case 5 — Privilege

ID	User	Akses	Expected
TC-013	Admin	User Management	Diizinkan
TC-014	Petugas	User Management	Ditolak
TC-015	Peminjam	User Management	Ditolak

44. IMPLEMENTASI WATERFALL

Tahap 1 — Analisis

Output:

- ✓ Identifikasi aktor
- ✓ Identifikasi kebutuhan
- ✓ Identifikasi fitur
- ✓ Hak akses
- ✓ Business process
- ✓ Non-functional requirement

Tahap 2 — Desain

Output:

- ✓ ERD
- ✓ Struktur database
- ✓ Flowchart
- ✓ Arsitektur MVC
- ✓ Struktur folder
- ✓ Desain interface
- ✓ Desain hak akses

Tahap 3 — Implementasi

Urutan coding:

1. Database
2. Core MVC
3. Routing
4. Authentication
5. Authorization
6. User
7. Category
8. Tool
9. Borrowing
10. Approval
11. Return
12. Fine
13. Activity Log
14. Report
15. Dashboard

Tahap 4 — Pengujian

```
Unit/Function Test
    ↓
Integration Test
    ↓
Black Box Test
    ↓
Privilege Test
    ↓
Database Transaction Test
    ↓
Final Test
```

Tahap 5 — Dokumentasi

Dokumentasi akhir terdiri dari:

1. Dokumentasi sistem
2. ERD
3. Flowchart
4. Struktur database
5. Dokumentasi fungsi/method
6. Screenshot aplikasi
7. Screenshot pengujian
8. Debugging

- 9. Bug yang belum diperbaiki
- 10. Rencana pengembangan

Hal tersebut selaras dengan hasil yang diminta soal berupa folder proyek, database `.sql`, ERD, deskripsi program, dokumentasi fungsi/prosedur, debugging, pengujian/tangkapan layar, dan laporan evaluasi.

45. ROADMAP PEMBUATAN APLIKASI

Untuk mengerjakan aplikasi secara nyata, saya menyarankan urutan berikut:

```
PHASE 1
Database
  ↓
PHASE 2
Core MVC
  ↓
PHASE 3
Authentication
  ↓
PHASE 4
Role & Middleware
  ↓
PHASE 5
Master Data
  ↓
PHASE 6
Peminjaman
  ↓
PHASE 7
Approval
  ↓
PHASE 8
Pengembalian
  ↓
PHASE 9
Denda
  ↓
PHASE 10
Activity Log
  ↓
PHASE 11
Laporan
  ↓
PHASE 12
Dashboard
  ↓
PHASE 13
```

Security Hardening

↓

PHASE 14

Testing

↓

PHASE 15

Dokumentasi

46. FITUR TAMBAHAN UNTUK MEMBUAT SISTEM LEBIH KOMPLEKS

Tanpa keluar dari kebutuhan soal, sistem dapat ditingkatkan dengan:

1. Dashboard berbeda berdasarkan role

Admin → Management

Petugas → Operasional

Peminjam → Transaksi pribadi

2. Pagination

Misalnya:

10 / 25 / 50 data per halaman

3. Search

Pencarian:

Kode alat

Nama alat

Kategori

Status

4. Filter laporan

Tanggal awal

Tanggal akhir

Kategori

Status

Peminjam

5. Notifikasi

Contoh:

"Peminjaman PJ-001 menunggu persetujuan."

"PJ-002 terlambat 3 hari."

"Pengembalian PJ-003 berhasil."

6. Audit Log

Semua aktivitas penting dicatat.

7. Soft Delete

Data penting tidak langsung dihapus permanen.

8. Database Transaction

Untuk menjamin konsistensi stok dan transaksi.

9. Role-Based Access Control

Setiap controller/method melakukan pemeriksaan hak akses.

10. Validasi stok

Tidak boleh terjadi:

```
available_qty < 0
```

47. KONSEP KEAMANAN

Struktur keamanan:

```
REQUEST
↓
CSRF Validation
↓
Authentication
↓
Authorization
↓
Input Validation
```

↓
Controller
↓
Prepared Statement
↓
Database

Untuk password:

```
$passwordHash = password_hash($password, PASSWORD_DEFAULT);
```

Untuk login:

```
password_verify($password, $user['password']);
```

Untuk database:

PDO + Prepared Statement

Jangan menggunakan:

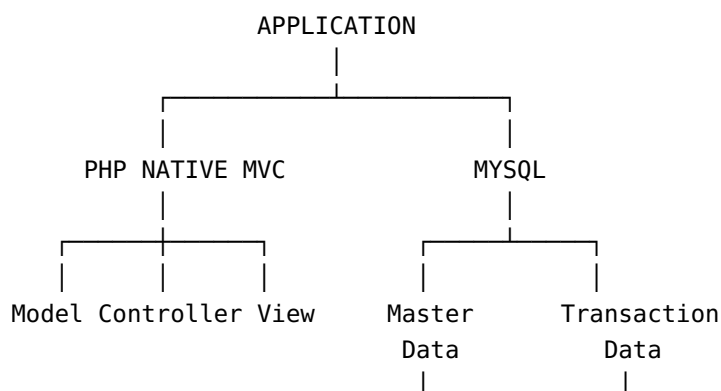
```
$sql = "SELECT * FROM users WHERE username = '$username'";
```

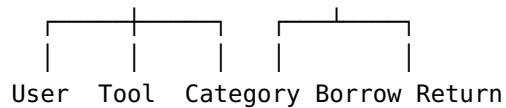
Gunakan prepared statement.

48. KESIMPULAN DESAIN

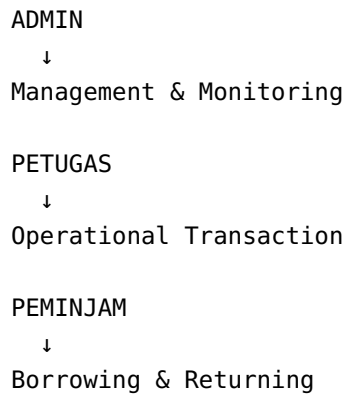
Dengan rancangan ini, aplikasi tidak hanya memenuhi fitur dasar UKK tetapi mempunyai arsitektur yang cukup kuat untuk dikembangkan menjadi aplikasi peminjaman alat yang kompleks.

Komponen utama sistem:

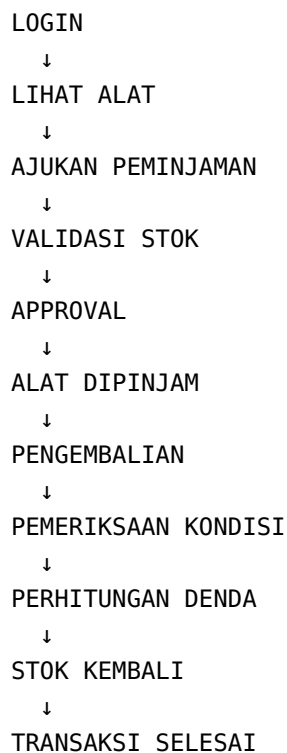




Sistem utama memiliki tiga aktor:



Dan siklus bisnis utama:



Dengan rancangan ini, tahap berikutnya sudah bisa masuk ke **implementasi nyata PHP Native MVC**, dimulai dari **database .sql lengkap + ERD final + struktur folder MVC + routing + authentication + middleware role**, kemudian dilanjutkan modul satu per satu sampai aplikasi selesai.

Saya juga bisa membuat **gambar ERD dan flowchart visual** dari rancangan ini agar siap dimasukkan ke laporan UKK.